

Choosing Features for Anomaly Detection

In Supervised Learning, the algorithm has labels (y) to guide it, so it can often learn to ignore irrelevant features on its own. However, in **Anomaly Detection**, the algorithm learns purely from unlabeled data. Therefore, **choosing the right features is critical** to the system's success.

1. Transforming Non-Gaussian Features

The anomaly detection algorithm assumes that your features follow a **Gaussian (Normal) Distribution** (the bell curve). If your data is "skewed" (not bell-shaped), the algorithm might perform poorly.

The Process:

1. **Plot a Histogram:** Use code (like `plt.hist`) to visualize your feature.
2. **Check Shape:** Does it look like a bell curve?
 - **If Yes:** Good to go.
 - **If No:** Apply a transformation to shape it into a bell curve.

Common Transformations:

If a feature x is skewed (e.g., has a long tail to the right), try these transformations to "squash" the tail:

- **Log:** $\log(x)$ (Note: if x can be 0, use $\log(x + c)$ where c is a small constant).
- **Roots:** $\sqrt{x}$ (or $x^{0.5}$), $x^{1/3}$, etc.
- **Exponentiation:** $x^{0.4}$, etc.

You can simply try different transformations in a notebook until the histogram looks roughly Gaussian. **Important:** Whatever transformation you apply to the training set, you must also apply to the Cross-Validation and Test sets.

2. Error Analysis for Anomaly Detection

If your model is not performing well on your Cross-Validation set, performing **Error Analysis** can help you find better features.

Scenario:

Your model fails to flag a specific anomalous example (x_{anom}) because it assigns it a high probability ($p(x) \geq \epsilon$).

The Solution:

1. **Inspect the Example:** Look closely at that specific x_{anom} .
 2. **Ask:** "Why is this anomalous? What is strange about it compared to the normal examples?"
 3. **Create a New Feature:** Try to design a new feature that takes on an unusually large or small value *only* for this anomaly.
- **Example:**
 - You are detecting fraud.
 - The model misses a fraudster because their "Transaction Count" (x_1) is normal.
 - **Inspection:** You notice this user types insanely fast.
 - **Action:** Add a new feature: "Typing Speed" (x_2). Now the anomaly stands out.
-

3. Combining Features (Feature Engineering)

Sometimes, an anomaly isn't obvious when looking at features individually, but becomes obvious when looking at the **relationship** between features.

Example: Data Center Monitoring

Imagine you are monitoring a computer server.

- x_1 : CPU Load
- x_2 : Network Traffic

The Anomaly: A machine is stuck in an infinite loop.

- It has **High CPU Load** (which is normal for a busy server).
- It has **Low Network Traffic** (which is normal for an idle server).
- However, the *combination* (High CPU with Low Traffic) is very strange.

The Fix: Create a new feature to capture this relationship.

- $x_{new} = \frac{\text{CPU Load}}{\text{Network Traffic}}$
 - For the anomalous machine, this ratio will be incredibly large, allowing the algorithm to flag it easily.
-

Summary of Course 3, Week 1

Congratulations on finishing the first week! You have now mastered two major Unsupervised Learning techniques:

1. **Clustering (K-Means):** For grouping data.
2. **Anomaly Detection:** For finding unusual events.